PRODUCT OVERVIEW · VERSION 1.1.1

# QA Scout
# Explore with a mission. Come back with evidence.

A free and open source Chrome extension for session-based exploratory testing. It records what happens while you explore and hands the team a ready-made report and bug.

**0**
network requests: your data stays on your computer

**4**
languages: pt-BR, pt-PT, English and Spanish

**176**
automated tests in the project itself

Thamires Azevedo · Senior QA · github.com/azevedothamirest/qa-scout

THE CONCEPT

# What is exploratory testing?

It means learning about the system, designing the tests and running them **at the same time**. Instead of following a fixed script, the tester investigates with a mission, watches the real behavior and picks the next step based on what they just saw.

### Scripted testing

Checks what we already know should happen. Good for regression and automation.

### Exploratory

Finds what nobody thought to write down: combinations, edge cases, real flows. That is where the most expensive bugs show up.

## Time-boxed sessions (SBTM)

**Session-Based Test Management**, by Jon and James Bach, adds structure without becoming a script. Each session has three parts:

1. **Charter:** the mission, in Elisabeth Hendrickson's format.
2. **Session:** an uninterrupted time box (30 to 90 min), logging bugs, questions, ideas, risks and observations.
3. **Debriefing:** review of the notes, time split into Test, Bug and Setup (TBS), and what is left for the next session.

Explore **the instant-payment checkout** with **an expired QR code, the back button and coupons** to discover **paid orders stuck as pending**.

### Time split (TBS)



Test 60% · Bug 25% · Setup 15%. Too much time on setup or bugs points to an unstable environment or a fragile product: a metric the whole team understands.

THE PROBLEM

# Exploratory testing is often invisible work

It finds the best bugs but leaves little trace: what was explored, for how long, with what evidence? And reproducing the bug later turns into archaeology.

THE SOLUTION

# QA Scout takes notes while you explore

### 📸 Automatic capture

Clicks, filled fields and opened pages become notes with screenshots, in Chrome's side panel. **Typed values are never read.**

### 🧯 Console and network errors

JavaScript errors, `console.error`, 4xx/5xx responses and connection failures become notes suggested as bug or risk.

### 🪜 Steps to reproduce

One click builds the list of actions that came before the bug.

### 🐞 Jira-ready bug

"Copy bug" produces title, environment, browser, page, steps, actual vs. expected and the errors from the same minute.

### 🃏 Heuristics deck

18 cards (Whittaker tours, data, interaction, context) for when you get stuck.

### 📄 Session report

Debriefing with TBS and a Markdown or PDF report to paste into Jira, Notion or Slack.

**You review everything.** Automatic notes arrive "to review": confirm, edit or delete. Only reviewed notes go into the report.

FOR THE WHOLE TEAM

# How each role uses it

Exploratory testing is not just for QA. With a short mission and 20 minutes, anyone on the team finds what scripted tests miss.

QA

### Sessions with mission and evidence

- Plan the sprint as charters by risk: what could cost the most?
- Use TBS in the debriefing to show where the time went.
- File bugs with steps and console errors attached, no more "can't reproduce".
- Turn what the session found into automated regression cases.

Dev

### Explore before opening the PR

- A 15-minute session on your own feature: "explore the new form with invalid data to discover missing validations".
- Bugs arrive with steps, environment, URL and the JavaScript error from the same minute: less back and forth.
- During PR review, explore the branch on staging and paste the report into the PR.

Product

### Validate the real experience

- Walk the journey as a customer before the release: "explore onboarding with a new user to discover where people get stuck".
- **Question** and **idea** notes become refinement questions and backlog items.
- The report shows coverage and risks in business language, great for the go/no-go call.

### 💡 Suggested ritual: a 30-minute bug bash

Each person (QA, dev, product, design) takes a different charter, everyone explores at the same time and the team merges the reports at the end. Fast, cheap and very revealing before a release.

GETTING STARTED

# Install in 1 minute

### 🧩 Chrome extension (recommended)

Install it from the Chrome Web Store, open the side panel from the compass icon and start a session. Chrome asks for site access on first use.

### 🖥️ Test browser (your Chrome stays untouched)

With Node.js 18+:

```
npx github:azevedothamirest/qa-scout https://your-system
```

Full walkthrough, source code and tutorial: **github.com/azevedothamirest/qa-scout** (see `docs/tutorial.md`).

WITH AI

# Claude explores with you (MCP)

QA Scout has an **MCP** server that lets Claude run the session in the test browser: it reads the screen, clicks, fills in, checks the captured errors, logs what it finds and hands back the report. The session stays in the history for you to review.

```
claude mcp add qa-scout -- npx -y github:azevedothamirest/qa-scout mcp
```

"Use QA Scout for a 20-minute session on https://staging.mystore.com: explore the checkout, with coupons and card payments, to discover wrong charges. Show me the report."

### 1. Reads

Understands the screen through the accessibility tree.

### 2. Explores

Clicks and types like a person; the capture records everything.

### 3. Logs

Logs bugs, risks and questions and wraps up with the report.

Runs in the terminal (Claude Code or Claude Desktop). The text of visited pages is sent to Claude: use only test environments and test data.

CAN I USE IT AT WORK?

# Privacy and security

- ✓ No server, no account, no telemetry: **zero network requests** (blocked by CSP `connect-src 'none'`).
- ✓ Data lives only in the browser's local storage; it leaves only when you copy or download something.
- ✓ Typed values are never read; tokens and passwords in URLs are masked.
- ✓ Open source (MIT license): everything it does can be audited.
- ✓ Material for your security team: `docs/seguranca.md`, terms of use and `SECURITY.md`.

# Summary

**QA**

Evidence and traceability for exploratory testing.

**Dev**

Reproducible bugs, with technical context.

**Product**

A real view of the experience and the risks.

**Explore with a mission. Come back with evidence.**

github.com/azevedothamirest/qa-scout · Thamires Azevedo · @qa.thami